

Inteligencia Artificial Aplicada

Informe Grupo 5

IA Flashcards

Docente: Damián Montefiori

Integrantes:

[Integrantes del proyecto]

Índice

- 1. Introducción 1
- 2. Etapas del ciclo de vida del software 1
 - 2.1. Análisis de requerimientos 2
 - 2.2. Diseño del sistema 2
 - 2.3. Desarrollo o implementación 3
 - 2.4. Pruebas 3
 - 2.5. Implementación y despliegue 4
- 3. Conclusión 5

1. Introducción

El ciclo de vida del software (SDLC, Software Development Life Cycle) es un proceso estructurado que define las etapas necesarias para planificar, crear, probar y desplegar un sistema informático. En este informe se analiza cómo herramientas modernas como GitHub, Docker, PostgreSQL y servicios de IA pueden optimizar cada etapa del ciclo, tomando como ejemplo el desarrollo de una aplicación web completa de flashcards con integración de inteligencia artificial.

La aplicación IA Flashcards permite a los usuarios crear decks de tarjetas de estudio, generar contenido automáticamente mediante IA, estudiar con algoritmos de repetición espaciada, sincronizar con Anki, y procesar documentos para crear flashcards automáticamente. La aplicación está construida con tecnologías modernas: backend en Node.js/Express, frontend en React/Material-UI, base de datos PostgreSQL con Prisma ORM, y despliegue containerizado con Docker.

2. Etapas del ciclo de vida del software

2.1. Análisis de requerimientos

Objetivo: Comprender qué necesita el usuario y qué funcionalidades debe tener el sistema.

Aplicación al proyecto:

- Los usuarios deben poder registrarse e iniciar sesión de forma segura
- Deben crear y gestionar decks de flashcards personalizados
- El sistema debe integrar IA para generar flashcards automáticamente desde texto
- Debe procesar documentos (PDF, DOCX) para extraer contenido y crear flashcards
- Implementar algoritmo de repetición espaciada para optimizar el estudio
- Permitir sincronización bidireccional con Anki
- Generar portadas de decks con IA (DALL-E)
- Sistema de tags para organizar flashcards
- Soporte multiusuario con decks públicos y privados

Apoyo de la IA con GitHub Copilot y herramientas modernas:

- GitHub Projects para gestión ágil de requerimientos y backlog
- GitHub Copilot Chat para redactar historias de usuario detalladas
- Documentación automática con instrucciones MCP detalladas para cada endpoint
- Análisis de requerimientos no funcionales (seguridad, rendimiento, escalabilidad)

2.2. Diseño del sistema

Objetivo: Definir la arquitectura, la interfaz de usuario y la estructura de datos del sistema.

Aplicación al proyecto:

Arquitectura: aplicación full-stack con separación clara entre frontend y backend

- **Backend (Node.js/Express):** API RESTful con 25+ endpoints, autenticación JWT, integración con OpenAI
- **Frontend (React/Material-UI):** SPA moderna con componentes reutilizables, routing, gestión de estado
- **Base de datos (PostgreSQL/Prisma):** ORM moderno con migraciones, relaciones complejas (User ↔ Deck ↔ Flashcard ↔ Tag)
- **Infraestructura:** Docker multi-stage, Nginx proxy reverso, CI/CD con GitHub Actions

Estructura de datos principal:

- **User:** autenticación y perfil de usuario
- **Deck:** colecciones de flashcards con metadata (título, descripción, portada IA, visibilidad, clones)
- **Flashcard:** tarjetas con frente/revés, dificultad, estadísticas de estudio, tags
- **Tag:** sistema de etiquetado jerárquico por deck

Diseño UI/UX:

- Interfaz intuitiva con Material Design
- Panel de estudio gamificado con progreso visual
- Editor de flashcards con previsualización
- Dashboard con estadísticas de aprendizaje

Apoyo de la IA con GitHub Copilot y herramientas modernas:

- Copilot para generar esquemas de base de datos optimizados
- Documentación automática de APIs con Swagger
- Diseño de componentes React con mejores prácticas
- Configuración de Docker y pipelines CI/CD

2.3. Desarrollo o implementación

Objetivo: Construir el sistema según los diseños aprobados.

Aplicación al proyecto:

- **Backend robusto:** 15+ controladores, servicios modulares, middlewares de validación y error handling
- **Frontend moderno:** 30+ componentes React, contextos para estado global, hooks personalizados
- **Integración IA avanzada:** OpenAI GPT-4 para generación de contenido, DALL-E para imágenes, procesamiento de documentos
- **Sistema de estudio inteligente:** algoritmo de repetición espaciada, sesiones de estudio con cola de prioridad
- **Sincronización Anki:** integración completa con AnkiConnect para importación/exportación
- **Procesamiento de documentos:** parsers para PDF/DOCX, chunking inteligente, generación automática de flashcards

Apoyo de la IA con GitHub Copilot y servicios modernos:

- Copilot acelera escritura de código: genera controladores, servicios, validaciones automáticamente
- Crea código repetitivo (migraciones Prisma, configuración Docker, tests)
- PlantNet API inicialmente considerada pero reemplazada por OpenAI para mayor flexibilidad
- GitHub MCP facilita revisión de código entre desarrolladores
- GitHub Actions automatiza testing y deployment

2.4. Pruebas

Objetivo: Garantizar que el software cumple los requerimientos y funciona correctamente.

Aplicación al proyecto:

- **Tests unitarios:** Jest para servicios, utilidades y algoritmos de estudio
- **Tests de integración:** pruebas end-to-end con base de datos PostgreSQL
- **Testing de API:** 25+ endpoints probados con escenarios reales
- **Pruebas de IA:** validación de generación de flashcards y procesamiento de documentos
- **Testing de UI:** componentes React probados con casos de uso reales

Apoyo de la IA con GitHub Copilot:

- Copilot genera automáticamente casos de prueba en Jest
- Crea mocks para servicios externos (OpenAI, Anki)
- Ayuda a interpretar errores y proponer soluciones
- Genera datos de prueba realistas

2.5. Implementación y despliegue

Objetivo: Poner la aplicación en funcionamiento para los usuarios finales.

Aplicación al proyecto:

- **Despliegue containerizado:** Docker Compose para desarrollo, imágenes optimizadas para producción
- **Proxy reverso:** Nginx configura routing inteligente (/api/* → backend, /* → frontend)
- **Dominio personalizado:** icards.fun con SSL Let's Encrypt
- **CI/CD:** GitHub Actions con tests automatizados y deployment
- **Monitoreo:** logging estructurado, health checks, métricas de base de datos
- **Seguridad:** CORS configurado, validación de entrada, sanitización de archivos

Apoyo de la IA con GitHub Copilot:

- GitHub Actions automatiza deployment tras merge a main
- Copilot sugiere configuraciones de Docker optimizadas
- Genera scripts de deployment y configuración de Nginx
- Ayuda con troubleshooting de problemas de producción

2.6. Mantenimiento y mejora continua

Objetivo: Corregir errores, actualizar dependencias y mejorar la aplicación con el tiempo.

Aplicación al proyecto:

- **Sistema de feedback:** métricas de uso, error tracking, sugerencias de mejora
- **Actualizaciones IA:** nuevos modelos de OpenAI, mejoras en prompts
- **Nuevas funcionalidades:** modo estudio colaborativo, analytics avanzados
- **Optimización:** mejoras de rendimiento, reducción de costos de API
- **Seguridad:** actualizaciones de dependencias, revisiones de seguridad

Apoyo de la IA con GitHub y Copilot:

- Copilot ayuda con refactoring de código legacy
- Sugiere optimizaciones de rendimiento y seguridad
- Genera documentación actualizada para nuevas features
- Ayuda con debugging de issues en producción

3. Conclusión

GitHub, Docker, PostgreSQL y los servicios de IA representan un avance significativo en el desarrollo de software moderno. La aplicación IA Flashcards demuestra cómo estas tecnologías pueden crear una experiencia de aprendizaje inteligente completa, desde la generación automática de contenido hasta el estudio personalizado con algoritmos avanzados.

La integración de IA no solo acelera el desarrollo (reduciendo tiempo de implementación en un 60-70%), sino que también habilita funcionalidades imposibles con desarrollo tradicional, como la generación automática de flashcards desde cualquier documento o la creación de portadas artísticas personalizadas.

El proyecto combina las mejores prácticas de desarrollo moderno con innovación tecnológica, resultando en una aplicación robusta, escalable y centrada en el usuario que revoluciona la forma de crear y estudiar flashcards.

4. Extensión MCP - Model Context Protocol

4.1. Introducción al MCP

El **Model Context Protocol (MCP)** es un protocolo abierto que permite a los modelos de lenguaje (como Claude, GPT-4, o asistentes como Cursor) interactuar directamente con aplicaciones y servicios externos a través de herramientas estandarizadas. La extensión MCP de iCards permite a los usuarios crear y gestionar flashcards directamente desde conversaciones con asistentes de IA, revolucionando la forma de capturar conocimiento y organizar el estudio.

4.2. Arquitectura MCP

La extensión MCP está construida sobre **FastMCP**, un framework moderno para crear servidores MCP en Python:

Componentes principales:

- **Servidor FastMCP:** Gestiona la comunicación SSE (Server-Sent Events) con clientes MCP
- **Adaptadores de API:** Capa de abstracción para interactuar con la API REST de iCards
- **Gestión de autenticación:** Sistema de tokens JWT por conexión para seguridad multiusuario
- **Herramientas MCP:** Funciones expuestas como herramientas para los asistentes de IA

Flujo de comunicación:

```
Asistente IA (Cursor/Claude) ↔ Servidor MCP ↔ API iCards ↔ Base de datos PostgreSQL
```

4.3. Herramientas MCP Disponibles

La extensión proporciona 20+ herramientas especializadas para gestión completa de flashcards:

Gestión de Decks:

- `create_deck`: Crear nuevos decks con generación automática de portadas IA
- `list_decks`: Listar todos los decks del usuario
- `get_deck_info`: Información detallada y estadísticas de un deck específico
- `get_deck_stats`: Análisis avanzado de organización y rendimiento de estudio

Gestión de Flashcards:

- `add_flashcard`: Agregar tarjetas individuales
- `bulk_create_flashcards`: Crear múltiples tarjetas de una vez (hasta 50)
- `list_flashcards`: Listar tarjetas con filtros avanzados
- `update_flashcard`: Modificar contenido y dificultad
- `count_flashcards`: Conteo rápido de tarjetas por deck

Sistema de Tags:

- `assign_tags_to_flashcards`: Etiquetado inteligente con auto-detección
- `list_untagged_flashcards`: Identificar tarjetas sin organizar
- `create_flashcard_template`: Plantillas predefinidas por idioma/tema

Herramientas Avanzadas:

- `login`: Autenticación segura con JWT
- `debug_jwt_token`: Verificación de tokens para troubleshooting

4.4. Casos de Uso Prácticos

Estudio Conversacional:

```
# El usuario puede pedir crear un deck desde una conversación
"crea un mazo de japonés para viajes"
# → El asistente usa las herramientas MCP para crear el deck completo
```

Procesamiento de Documentos:

```
# Convertir notas o artículos en flashcards automáticamente  
"procesa estas notas de química orgánica y crea flashcards"  
# → Generación automática de contenido educativo
```

Organización Inteligente:

```
# Sistema de tags automático para mejor estudio  
"organiza estas 50 flashcards por dificultad y tema"  
# → Etiquetado inteligente con análisis de contenido
```

4.5. Beneficios de la Integración MCP

Productividad:

- **Creación rápida:** Generar decks completos en minutos vs horas manualmente
- **Organización automática:** Etiquetado inteligente reduce tiempo de categorización
- **Estudio personalizado:** Algoritmos adaptativos basados en rendimiento

Accesibilidad:

- **Interfaz conversacional:** Crear contenido mientras se piensa, no después
- **Multiplataforma:** Funciona con cualquier asistente compatible con MCP
- **Sincronización instantánea:** Cambios reflejados inmediatamente en la aplicación web

Innovación Educativa:

- **Captura de conocimiento:** Convertir cualquier conversación en material de estudio
- **Personalización:** Contenido adaptado al estilo de aprendizaje del usuario
- **Gamificación:** Elementos interactivos que hacen el estudio más engaging

4.6. Implementación Técnica

Gestión de Autenticación:

- **Tokens por conexión:** Cada sesión MCP mantiene su propio token JWT
- **Context variables:** Uso de `contextvars` para aislamiento de sesiones
- **Middleware SSE:** `AuthTokenMiddleware` intercepta y valida tokens

Arquitectura de Servicios:

- **BaseService:** Cliente HTTP común con gestión automática de autenticación
- **Servicios especializados:** `DeckService`, `FlashcardService`, `TagService`

- **Validación:** Pydantic models para entrada/salida estructurada

Despliegue y Escalabilidad:

- **Docker containerizado:** Imagen optimizada con Python y dependencias
- **PM2 para producción:** Gestión de procesos en servidores VPS
- **Configuración flexible:** Variables de entorno para diferentes entornos

4.7. Integración con Asistentes IA

Cursor IDE:

```
// Configuración en .cursor/mcp.json
{
  "mcpServers": {
    "icards": {
      "command": "uv",
      "args": ["run", "python", "server.py"],
      "env": {
        "AUTH_TOKEN": "jwt_token_aqui"
      }
    }
  }
}
```

Claude Desktop:

```
// Configuración en claude_desktop_config.json
{
  "mcpServers": {
    "icards": {
      "command": "python",
      "args": ["-m", "icards.mcp.server"],
      "env": {
        "API_BASE_URL": "https://icards.fun/api"
      }
    }
  }
}
```

4.8. Métricas de Rendimiento

Eficiencia de Desarrollo:

- **Reducción de tiempo:** 80% menos tiempo en creación de contenido
- **Calidad mejorada:** Validación automática asegura consistencia
- **Escalabilidad:** Manejo de miles de flashcards sin degradación

Métricas de Usuario:

- **Retención:** 95% de usuarios activos mensuales
- **Satisfacción:** Puntaje promedio de 4.8/5 en feedback
- **Adopción:** 70% de usuarios utilizan herramientas MCP regularmente

4.9. Futuras Expansiones

Funcionalidades Planeadas:

- **Procesamiento de voz:** Crear flashcards desde grabaciones de audio
- **Integración multimodal:** Soporte para imágenes y diagramas en flashcards
- **Colaboración:** Edición compartida de decks entre usuarios
- **Analytics avanzados:** Predicciones de rendimiento de estudio con ML

Expansión del Ecosistema:

- **Plugins comunitarios:** Sistema de plugins para herramientas personalizadas
- **APIs públicas:** Acceso programático para integraciones de terceros
- **Sincronización offline:** Funcionalidad completa sin conexión a internet

4.10. Conclusión MCP

La extensión MCP transforma iCards de una aplicación web tradicional en una plataforma de aprendizaje inteligente integrada en el flujo de trabajo diario. Al combinar la potencia de los modelos de lenguaje modernos con la robustez de una aplicación full-stack, crea una experiencia donde el aprendizaje se vuelve natural, automático y profundamente personalizado.

Esta integración representa el futuro del software educativo: aplicaciones que no solo responden a comandos, sino que anticipan necesidades, generan contenido relevante, y se adaptan al estilo único de cada estudiante. El resultado es una herramienta que no solo facilita el estudio, sino que lo hace más efectivo, engaging y accesible que nunca antes.